॥श्री गणेशाय नमः॥

Graphs:

Representation → Adjacency matrix (dense) : $O(V^2)$ space

Representation → Adj. list (sparse) : $O(V+E)$ space

For n vertices:
- max # of edges = ${}^nC_2$
- min edges to connect all = $n-1$ (spanning tree)

of spanning trees for K_n = n^{n-2}
↳ complete graph with n vertices

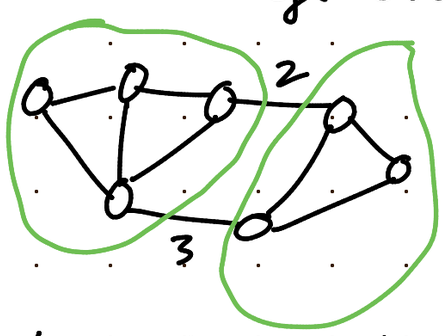
Kirchoff's theorem: For any non-complete graph,
- Draw adj. matrix.
- Replace diagonals with degree.
- Replace non-diagonal (1)s with (-1)s.

of spanning trees = cofactor of any element

Spanning trees: subgraph that is tree & includes all vertices.

MST: smallest weight

↳ cut property: smallest edge crossing any cut must be a part of all MSTs.
↳ partition of vertices into 2 non-empty sets



cut
∴ 2 is part of MST.

↳ cycle property: largest edge in cycle is never in MST.

Kruskal's algorithm: Pick edges greedily provided they do not form a cycle.

$T = O(E \log V)$

Prim's algo: Pick the smallest connected vertices greedily.

If edge weights are unique → MST is unique.

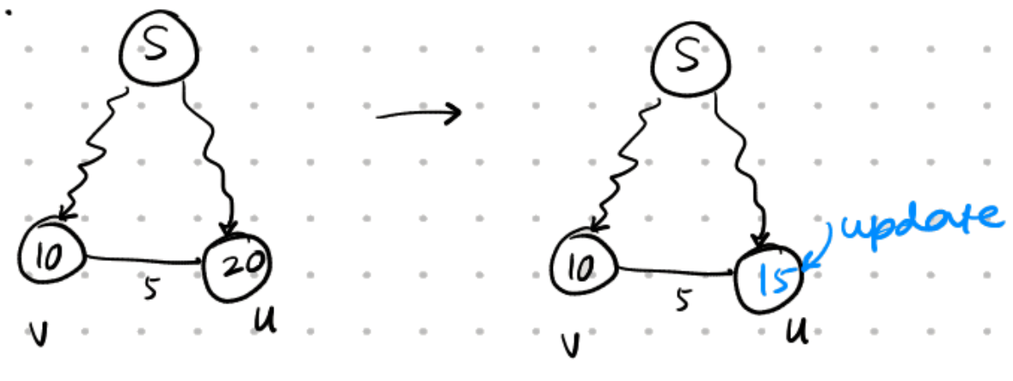
Time complexity →
- $O(V^2)$ if adj. matrix
- $O(E \log V)$ binary heap + adj list
- $O(E + V \log V)$ Fibonacci heap + adj list

Dijkstra's

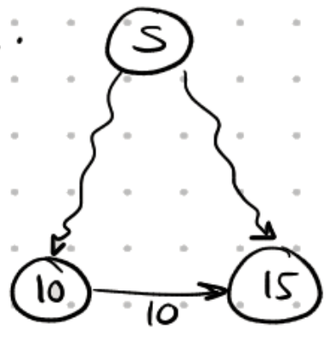
Dijkstra's algorithm

Relaxation : If $d(u) > d(v) + c(v,u) \Rightarrow d(u) = d(v) + c(u,v)$

e.g.

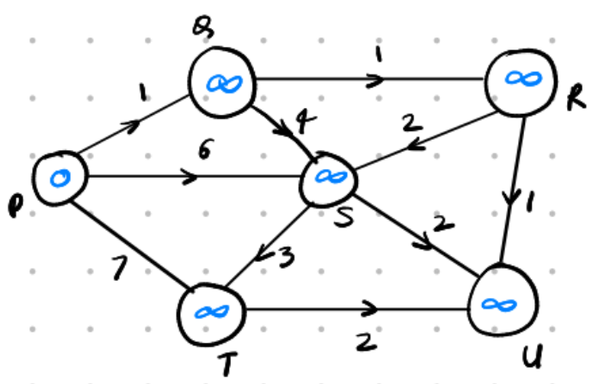


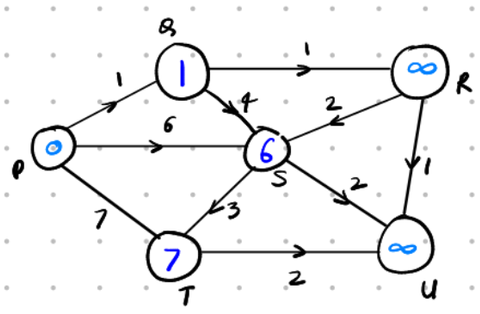
ex.2.



here relaxation is useless as $10 + 10 = 20$ worse than 15.

here we'll relax to get a better distance to u.



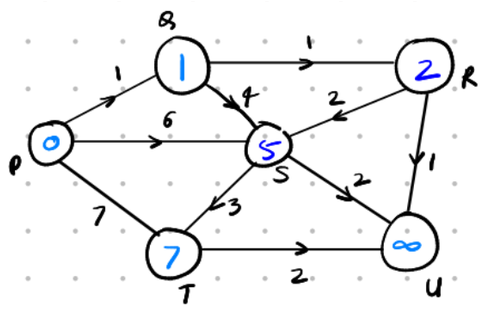


P

$P \xrightarrow{1} Q$ 1

$P \xrightarrow{7} T$ 7

$P \xrightarrow{6} S$ 6

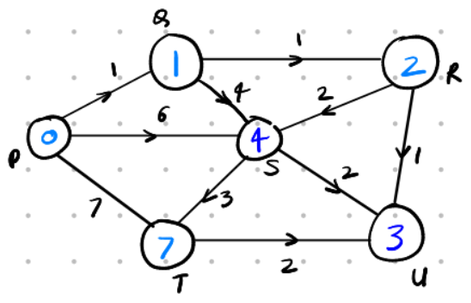


Q

Q S T
1 6 7

$Q \xrightarrow{1} R$ ✓

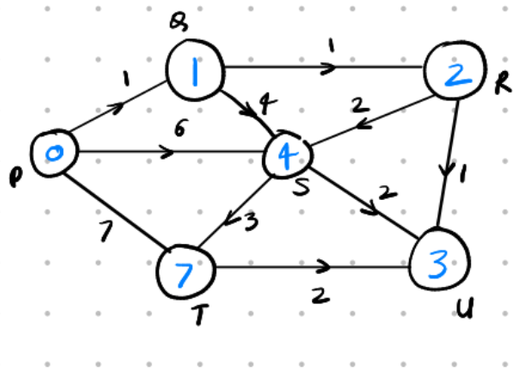
$Q \xrightarrow{4} S$ 5 ✓



R S T
2 5 7 R

$R \to U$ 3 ✓

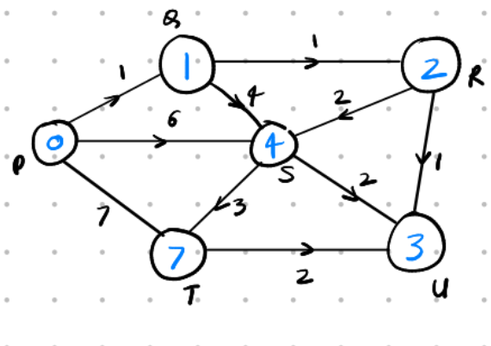
$R \to S$ 4 ✓



S T U
4 7 3
1

U

no outgoing edges



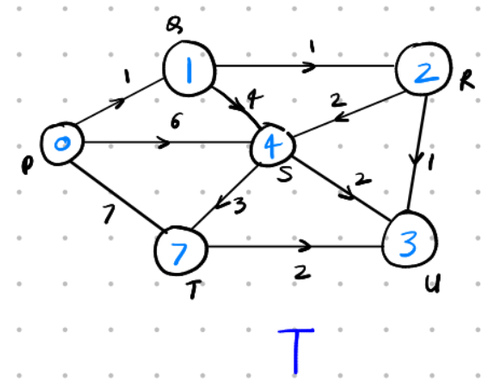
S T
4 7
↗

S

$S \to T$ $4+3=7$ same. no update

$S \to U$
↑ already done

No relaxⁿs



T

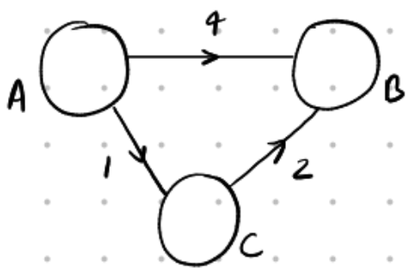
order : PQRUST

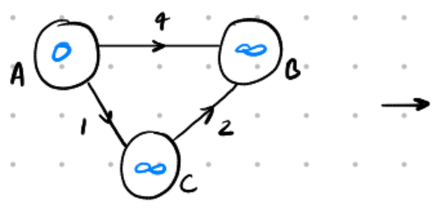
NOTE: once you've relaxed for a vertex, its value is not changed later.

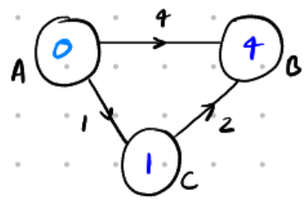
Bellman ford algorithm

(V-1)E relaxations.

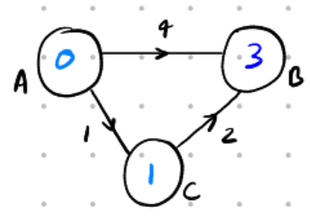
$T = O(V * E)$





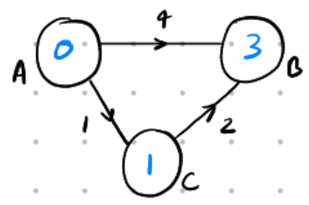


1st pass



2nd pass

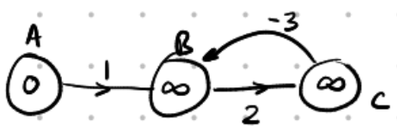
(n-1) passes to find the shortest paths.

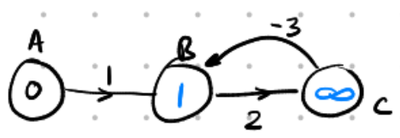


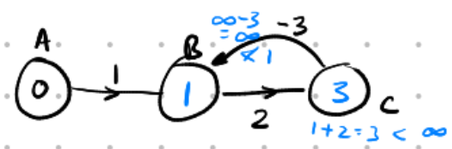
No weight change.

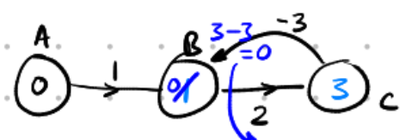
3rd pass

1 pass to check for -ve cycles.







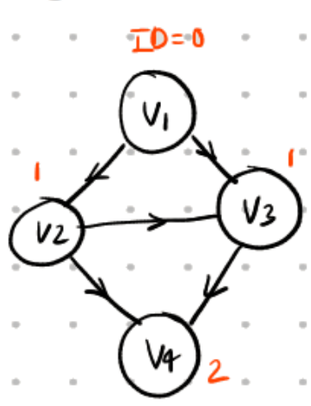


relaxing in the 3rd pass changes distances.

⇒ Theres a -ve cycle

Directed acyclic graph

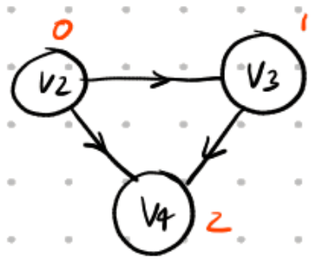
Topological sort on DAG



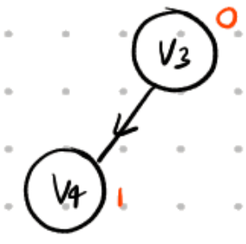
We start with nodes with indegree = 0.

V_1

We delete it and its outgoing edges.



V_2 next-

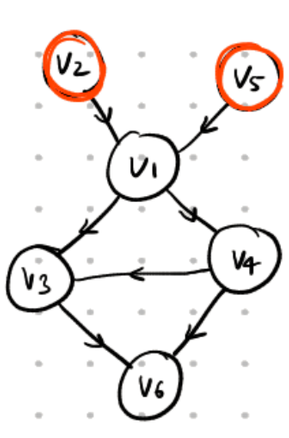


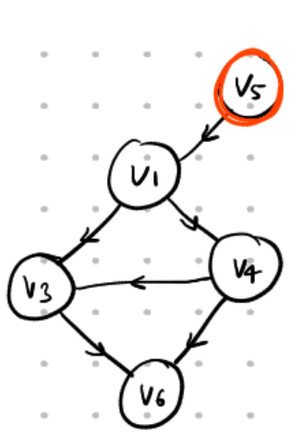
V_3

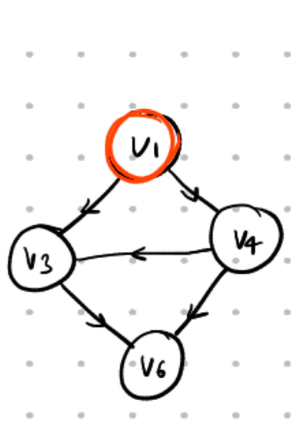


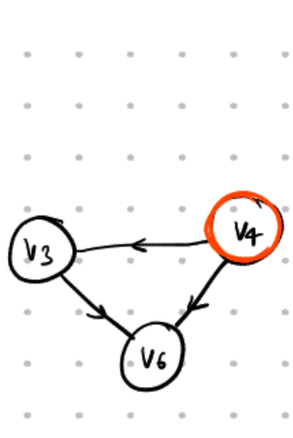
V_4

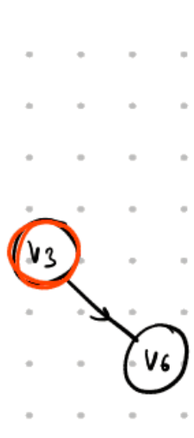
order: V_1, V_2, V_3, V_4 (not necessarily unique)













order: $V_2\ V_5\ V_1\ V_4\ V_3\ V_6$

OR

$V_5\ V_2\ V_1\ V_4\ V_3\ V_6$

Step 1: Identify vertices that have no incoming edges. ← $O(V^2)$ (V 1 time)

Step 2: Delete a vertex with indegree 0 and all its outgoing edges from the graph. ← $O(E)$

Repeat steps 1 and 2 till the graph is empty.

Can be reduced:
Whenever edge is deleted, put nodes of indegree 0 into a queue.
Then picking a node from that would be O(1).
So just the initial O(V) would be needed.

$T = O(V+E)$

DFS

```
DEPTH_FIRST_TRAVERSAL(G, n):
    for i = 1 to n do
        visited[i] = false
    for i = 1 to n do
        if visited[i] == false then
            DEPTH_FIRST(i)
```

* (u) ⟶ (v)

for any directed edge (u,v),

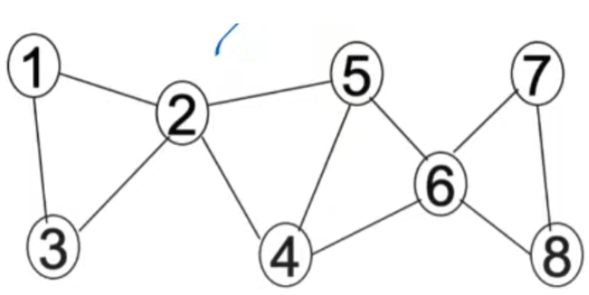
finish time (v) < finish time (u)

* Back-edge ≡ cycles.

* Articulation point / cut point : Its removal disconnects G.

Brute-force approach: remove each.

Using DFS:



for DFS tree:

→ Root of tree can be AP. ✓ always (provided root has >1 child)

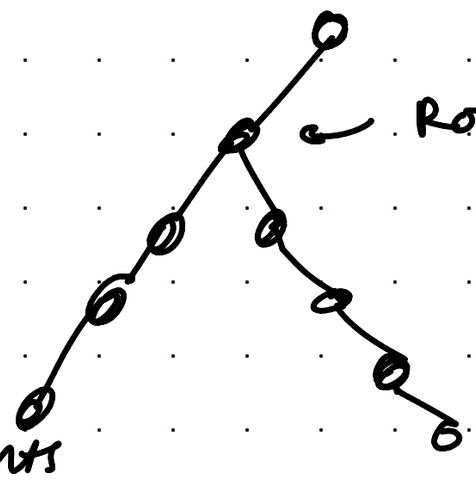
→ Leaf of tree can be AP. ✗ never

(Draw DFS tree



← NOT AP (its descendents are connected to its ancestors)

← AP (its desc. are not connected to its ancestors)



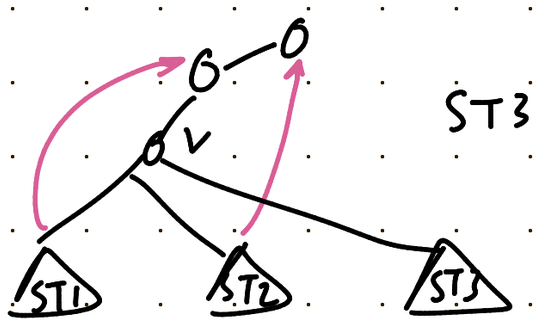
← Roots* like these are also APs

(5 is conn. to 2)

An internal node is AP if none of its descendants is connected to any of its ancestors.

To check if an internal node is AP:

check if there is a subtree that does not have backedge.



ST3 does not have a backedge.

∴ V is AP.

Space: O(n) (for visited + stack both)

Time:

O(V) to initialize visited
+ O(E) for adj list

O(V) to init visited
+ $O(V^2)$ for adj matrix

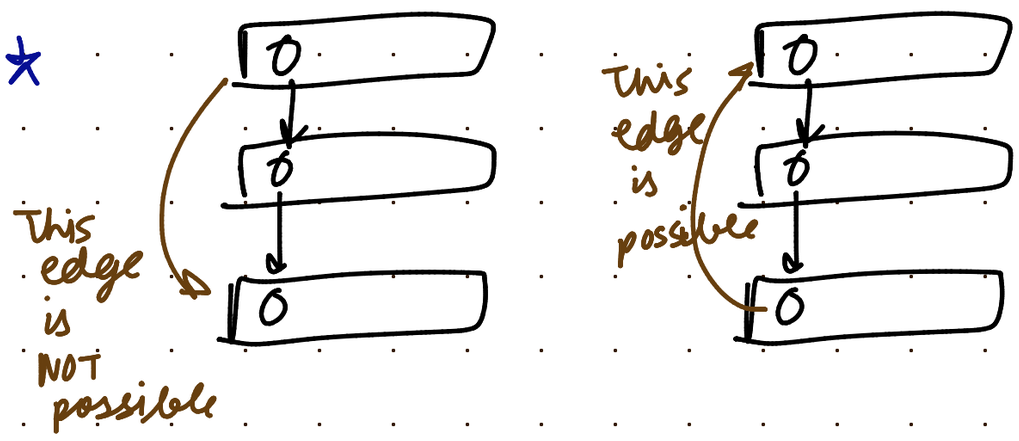
Time = $O(V+E)$

OR

$O(V^2)$

BFS

For directed graph:

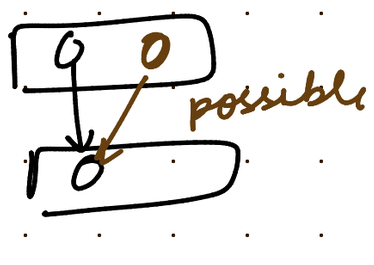


skip level edges are present only in directed graphs.
only for lower to upper levels.

within level edges: yes (for both directed and undirected graph)

skip edges
↳ for directed graphs (lower to higher level)

continuous level (apart from tree edges)



yes.

cross-edge
- within-level ✓
- continuous-level ✓

Shortest path using BFS:
For unweighted graph, shortest path = path in BFS tree

* For undirected graph, if no cross edges (only tree edges) ⟶ bipartite graph.
If cross-edge in undirected graph ⟶ may/may not be bipartite
A graph is not bipartite ⟷ odd-length cycle.

```
BREADTH_FIRST(v):
    -- Graph G, arr visited[]
    -- visited[] is initially all false
    u = v, visited[v] = true
    loop:
        for all vertices w in G adjacent to u do
            if visited[w] == false then
                enqueue(w)
                visited[w] = true
            if queue is empty then
                break
            u = dequeue()
```

Space:
- Visited: O(n)
- Queue: O(n)

Time:
- To initialize visited[]: O(V)
- To visit all adj lists: O(E)
- Total
- = O(V+E)

- To visit all elements in matrix: $O(V^2)$
- Total
- = $O(V^2 + V)$
- = $O(V^2)$

```
BREADTH_FIRST_TRAVERSAL(G, n):
    for i = 1 to n do
        visited[i] = false
    for i = 1 to n do
        if visited[i] == false then
            BREADTH_FIRST(i)
```

search ≠ traversal